МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»
(Университет ИТМО)

Образовательная программа ”Веб-технологии”
Направление подготовки (специальность) 09.04.04 Программная инженерия

Отчет
о производственной, технологической (проектно-технологической)
практике

по теме:
РАЗРАБОТКА ИССЛЕДОВАТЕЛЬСКОЙ ПЛАТФОРМЫ ДЛЯ АНАЛИЗА ОПТИМИЗАЦИИ, ПРОИЗВОДИТЕЛЬНОСТИ И ДОСТУПНОСТИ NO-CODE И LOW-CODE ВЕБ-ПРИЛОЖЕНИЙ

Обучающийся:
группа P4209 *Е.А. Стуков*

Научный руководитель:
доцент *И.Б. Государев*

Ответственный за практику (куратор):
преподаватель практики факультета ПИиКТ *О.В. Пархимович*

Санкт-Петербург
2026

СОДЕРЖАНИЕ

1 ВВЕДЕНИЕ 3

2 ОПИСАНИЕ ПРОЦЕССА ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ 6

2.1 Постановка задачи и формирование требований 6

2.2 Проектирование общей архитектуры решения 6

2.3 Реализация управления тестируемыми платформами 7

2.4 Конвейер автоматизированного сбора метрик 8

2.5 Хранение результатов и обеспечение воспроизводимости 9

2.6 Визуализация и сравнительный анализ 10

2.7 Апробация прототипа 12

3 ЗАКЛЮЧЕНИЕ 13

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 14

1 ВВЕДЕНИЕ

Актуальность темы обусловлена быстрым ростом числа no-code и low-code веб-приложений, которые создаются и развиваются без (или с минимальным объемом) традиционной разработки. Такие решения ускоряют вывод продукта на рынок, но одновременно усложняют контроль качества: внутренняя архитектура платформ часто частично скрыта, стек неоднороден, а поведение приложения определяется большим числом конфигураций, интеграций и сторонних компонентов.

Для пользователей и бизнеса критичны три группы характеристик: **оптимизация** (эффективность загрузки и использование ресурсов), **производительность** (время отклика и стабильность работы интерфейса) и **доступность** (устойчивость к отказам и доступность функциональности). На практике оценка этих характеристик обычно выполняется разрозненными инструментами и по разным методикам: синтетическое тестирование (лабораторные замеры), нагрузочное тестирование, мониторинг доступности, анализ клиентских метрик (RUM), аудит веб-производительности и т.д. Это приводит к тому, что результаты трудно сопоставимы между собой, а воспроизводимость экспериментов и повторное сравнение версий приложения требуют значительных ручных усилий.

Таким образом, актуальной задачей является разработка **исследовательской платформы**, которая обеспечивает единый процесс проведения экспериментов и получения сопоставимых метрик для no-code и low-code веб-приложений. Такое решение позволяет (1) стандартизировать методики измерений, (2) собирать метрики сразу по нескольким направлениям и инструментам в едином формате, (3) повторять эксперименты при изменении конфигурации, версии приложения или внешних условий, а также (4) выявлять компромиссы между оптимизацией, производительностью и доступностью.

Цель научно-исследовательской работы: разработать исследовательскую платформу для анализа оптимизации, производительности и доступности no-code и low-code веб-приложений, обеспечивающую сбор и агрегацию метрик по различным параметрам и методикам с возможностью воспроизводимых экспериментов и последующего сравнения результатов.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. **Определить функциональные требования и сценарии использования**. Зафиксировать целевые пользовательские сценарии (добавление платформ, запуск измерений, просмотр отчетов, сравнение результатов) и ограничения экспериментов (внешние URL, локальные приложения, Docker-сервисы).
2. **Реализовать управление платформами**. Реализовать добавление/редактирование/удаление платформ, поддержку нескольких типов развертывания (внешний URL, Docker, локальный порт), категоризацию и валидацию данных.
3. **Реализовать конвейер автоматизированного сбора метрик**. Реализовать поэтапный запуск измерений и сбор метрик по нескольким методикам: (1) аудит Google Lighthouse, (2) сбор через Performance API, (3) проверка доступности (availability check) с измерением времени отклика.
4. **Реализовать хранение и версионирование результатов экспериментов**. Организовать локальное хранение данных о платформах и собранных метриках в IndexedDB (включая связи «платформа — серия измерений», временные метки и каскадное удаление).
5. **Реализовать визуализацию и отчетность**. Реализовать отображение агрегированных оценок и детальных метрик (в т.ч. списков проблем Lighthouse), а также представление данных в виде графиков и таблиц.
6. **Реализовать сравнение платформ**. Реализовать сравнительный анализ по категориям метрик с определением лидеров и возможностью сопоставления результатов между платформами.
7. **Провести апробацию на наборе платформ**. Собрать метрики для нескольких low-code/no-code платформ (и/или конфигураций), оценить воспроизводимость и продемонстрировать получение метрик сразу по разным параметрам и методикам.
8. **Оформить отчётные документы** в соответствии с требованиями.
    a) Должно быть подробное описание выполнения задач по этапам.

б) Оформление отчета должно быть выполнено в соответствии с методическим пособием методическим пособием.
в) Структура документа: титульный лист, введение, основная часть, заключение.
г) В основной части подробно описывается выполнение задач этапов 1-7 данного индивидуального задания.

9. **Получить отзыв с оценкой** у руководителя практики в модуле «Практики». В случае возникновения вопросов по оценке явиться на дифференцированный зачет в установленные в ИСУ даты.

2 ОПИСАНИЕ ПРОЦЕССА ВЫПОЛНЕНИЯ ИНДИВИДУАЛЬНОГО ЗАДАНИЯ

2.1 Постановка задачи и формирование требований

В рамках производственной, технологической (проектно-технологической) практики была выполнена разработка исследовательской платформы-песочницы, предназначенной для анализа, мониторинга и сопоставления характеристик no-code и low-code веб-приложений. Практическая значимость данной задачи определяется тем, что экосистема low-code и no-code решений растет, тогда как воспроизводимая оценка качества создаваемых приложений затруднена из-за высокой доли конфигурационных параметров, неоднородности развертываний и ограниченной наблюдаемости внутренних компонентов платформ.

Требования к разрабатываемому прототипу формировались с ориентацией на исследовательский сценарий. Предполагалось обеспечить единый процесс постановки эксперимента, автоматизированный запуск измерительных процедур, сохранение результатов и дальнейший сравнительный анализ. Отдельно учитывалась необходимость поддержки нескольких способов подключения тестируемых объектов, включая внешние демонстрационные стенды, локально запущенные приложения и сервисы в контейнерном окружении. Таким образом, требования включали обеспечение сопоставимости результатов в рамках единого формата данных, а также практическую применимость решения для серийных измерений и выявления регрессий между версиями.

2.2 Проектирование общей архитектуры решения

Архитектура разрабатываемой платформы была спроектирована как клиент-серверная система с разделением ответственности между пользовательским интерфейсом и вычислительно емкими процедурами аудита. Пользовательская часть реализована в виде одностраничного веб-приложения, обеспечивающего управление набором тестируемых платформ, запуск измерений, просмотр детализированных отчетов и модуль сравнительного анализа. Серверная часть предназначена для выполнения аудита Google Lighthouse

в режиме headless, поскольку проведение такого аудита требует управляемого окружения браузера и устойчивого исполнения на стороне, независимой от пользовательского интерфейса.

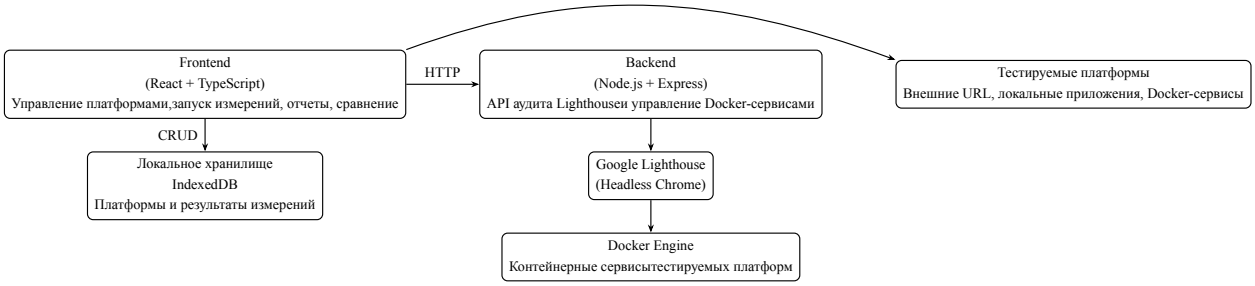


Рисунок 1 — Схема архитектуры исследовательской платформы-песочницы

Для хранения данных о тестируемых объектах и результатах измерений было выбрано локальное персистентное хранилище IndexedDB. Данное решение позволяет фиксировать результаты экспериментов без необходимости инфраструктуры централизованной базы данных, что соответствует формату исследовательского прототипа и обеспечивает воспроизводимость серий замеров в рамках одной рабочей среды. Источником истины для пользовательского интерфейса выступает локальная база, из которой выполняются загрузка списка платформ и извлечение последних измерений.

2.3 Реализация управления тестируемыми платформами

В ходе разработки был реализован модуль управления платформами, который обеспечивает полный жизненный цикл объекта исследования, включая создание, редактирование и удаление записей. Для каждой платформы фиксируется идентификатор, отображаемое имя, описание и категория, отражающая типологические особенности low-code решения. Дополнительно поддерживаются параметры развертывания, которые определяют способ обращения к тестируемому приложению и, при необходимости, позволяют контролировать его состояние. В частности, предусмотрены режимы работы с внешними URL, локальными приложениями по известному порту и сервисами, управляемыми посредством Docker. Реализация такого подхода необходима для обеспечения единых интерфейсов запуска измерений при различной природе тестируемых объектов.

2.4 Конвейер автоматизированного сбора метрик

Ключевым результатом практики является реализация конвейера автоматизированного сбора метрик, объединяющего несколько методик измерений в рамках единой процедуры. В качестве первой методики используется аудит Google Lighthouse, который предоставляет стандартизированные оценки и набор детальных показателей, связанных с производительностью, доступностью, SEO и соответствием лучшим практикам. Данные Lighthouse позволяют интерпретировать качество веб-приложения с позиций общепринятых веб-метрик и выявлять проблемные места в виде перечня неуспешных проверок.

В качестве второй методики применяется сбор показателей на основе Web Performance API, что позволяет дополнить результаты аудита фактическими временными характеристиками загрузки и подготовки документа к взаимодействию. Такой подход обеспечивает более прикладной взгляд на поведение приложений в условиях выполнения в браузере и повышает полноту наблюдения за динамикой загрузки при изменении конфигурации платформ или условий выполнения.

В качестве третьей методики реализована проверка доступности, включающая фиксацию статуса доступности и времени отклика. Данная процедура обеспечивает базовую оценку эксплуатационных характеристик и позволяет рассматривать качество приложения не только как набор клиентских метрик, но и как устойчивость при обращении по заданному адресу. Последовательное выполнение указанных методик организовано в виде этапов с контролем прогресса, что повышает удобство проведения эксперимента и снижает вероятность неконсистентного состояния при частичном завершении измерений.

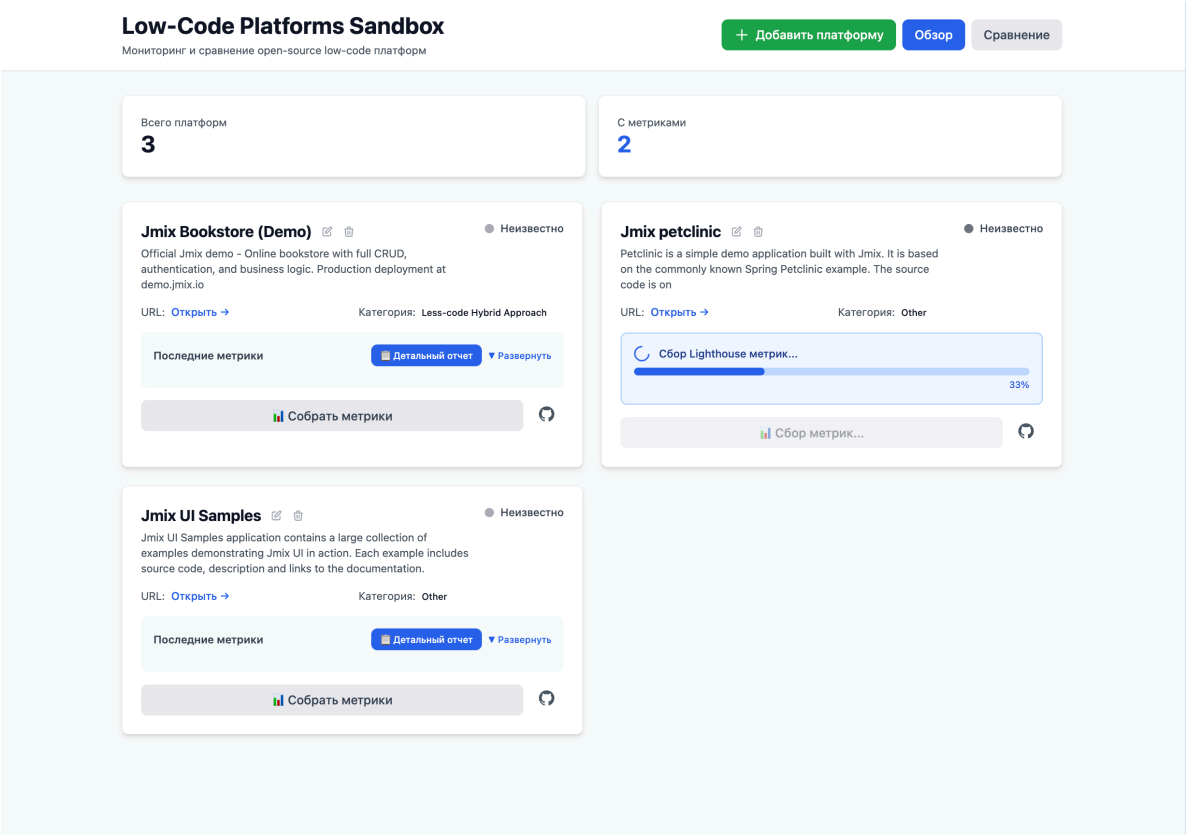


Рисунок 2 — Интерфейс платформы в процессе автоматизированного сбора метрик для выбранного проекта

2.5 Хранение результатов и обеспечение воспроизводимости

Результаты выполнения конвейера формируются в виде записи измерений, включающей временную метку, ссылку на тестируемую платформу и набор собранных показателей. Для обеспечения сопоставимости результатов все категории метрик сохраняются в едином формате, что упрощает последующий анализ и визуализацию. Персистентность результатов в IndexedDB обеспечивает накопление истории измерений, что позволяет сопоставлять результаты по времени и выполнять повторные серии замеров без утраты данных при перезапуске пользовательского интерфейса.

В процессе проектирования отдельно учитывалась устойчивость к частичным ошибкам измерений. В ряде случаев сбор отдельных категорий метрик может быть ограничен политиками происхождения или временными сбоями доступности. В связи с этим реализация ориентирована на деградацию без остановки эксперимента, при которой фиксируются доступные показатели и сохраняется контекст выполнения. Такой подход позволяет не терять

результаты целиком и обеспечивает корректность дальнейшего анализа на основе доступного множества метрик.

2.6 Визуализация и сравнительный анализ

Для поддержки аналитического сценария реализованы средства визуализации и сравнения результатов. Детализированный отчет по одной платформе предоставляет агрегированные оценки и набор подробных показателей, включая диагностическую информацию по Lighthouse. Для многоплатформенного анализа реализован модуль сравнения, ориентированный на выявление различий между решениями по ключевым категориям и на определение лидирующих платформ по отдельным группам показателей.

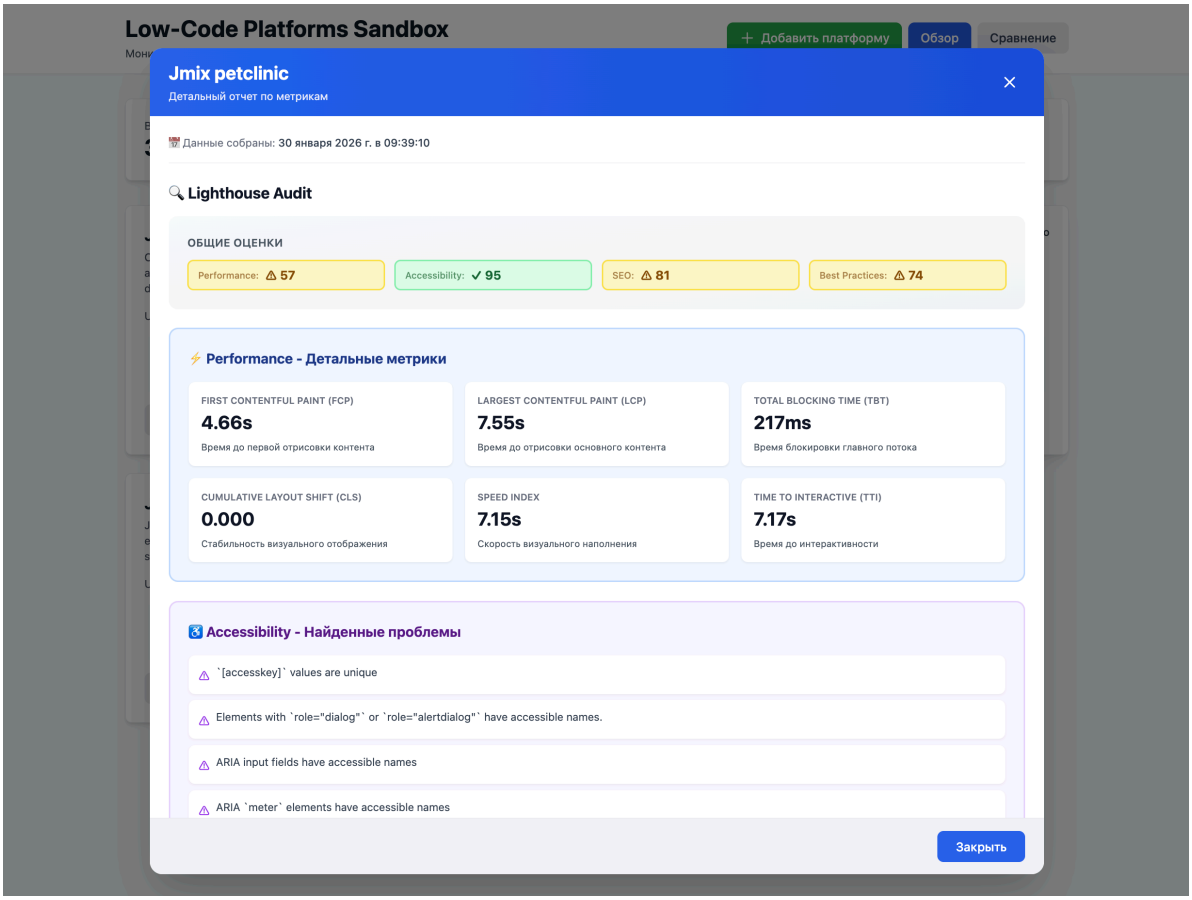


Рисунок 3 — Пример визуализации собранных метрик по выбранной платформе

Таким образом, итоговое решение обеспечивает получение метрик сразу по нескольким параметрам и методикам в рамках единого эксперимента. Это позволяет рассматривать характеристики оптимизации, производительности и доступности как взаимосвязанные показатели и проводить сопоста-

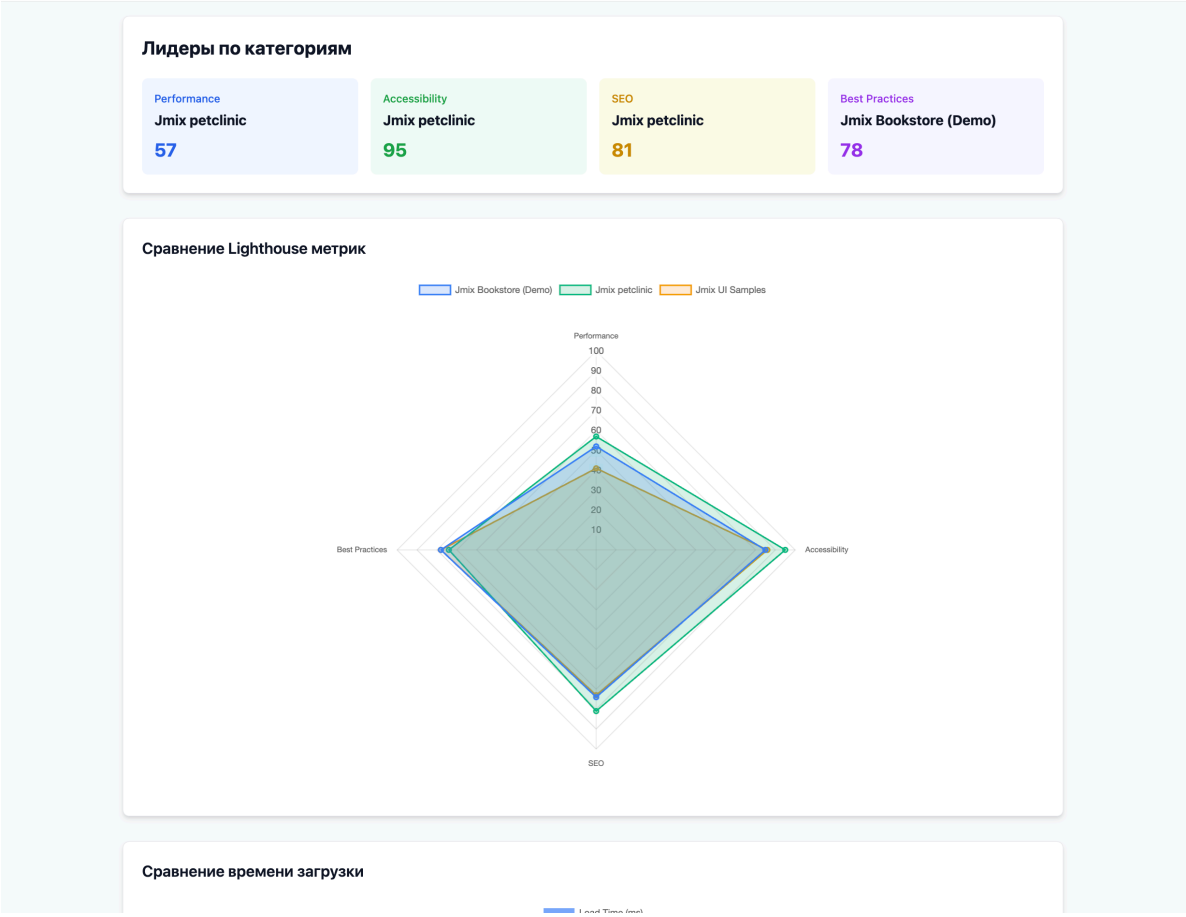
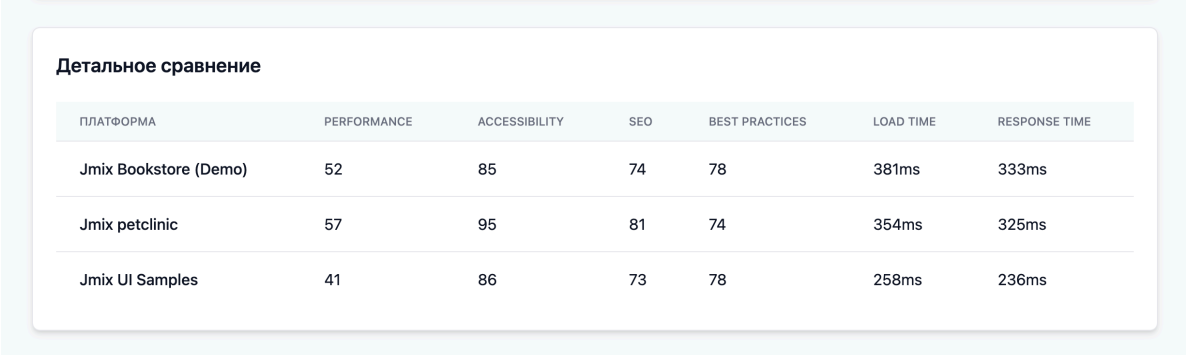


Рисунок 4 — Пример графического сравнения метрик между тестируемыми платформами (часть 1)



Детальное сравнение

ПЛАТФОРМА	PERFORMANCE	ACCESSIBILITY	SEO	BEST PRACTICES	LOAD TIME	RESPONSE TIME
Jmix Bookstore (Demo)	52	85	74	78	381ms	333ms
Jmix petclinic	57	95	81	74	354ms	325ms
Jmix UI Samples	41	86	73	78	258ms	236ms

Рисунок 5 — Пример графического сравнения метрик между тестируемыми платформами (часть 2)

вимый анализ между различными low-code и no-code решениями при минимальном объеме ручных операций.

2.7 Апробация прототипа

Проверка работоспособности прототипа выполнялась на наборе платформ, подключаемых различными способами. В ходе апробации подтверждена корректность сценариев управления платформами, воспроизводимость последовательности этапов сбора метрик и сохранение результатов измерений. Полученные данные продемонстрировали возможность формировать сопоставимые отчеты и выполнять сравнительный анализ в рамках единого интерфейса, что соответствует исходной постановке задачи.

3 ЗАКЛЮЧЕНИЕ

В ходе работы был разработан прототип исследовательской платформы-песочницы, предназначенной для анализа оптимизации, производительности и доступности no-code и low-code веб-приложений, а также для последующего сопоставления результатов измерений между различными платформами.

В процессе выполнения практики была проведена постановка задачи и сформированы требования, ориентированные на воспроизводимый исследовательский сценарий. На этапе проектирования была определена клиент-серверная архитектура решения с разделением ответственности между пользовательским интерфейсом и серверным модулем, выполняющим вычислительно емкий аудит Google Lighthouse. Для хранения сведений о тестируемых платформах и результатов экспериментов было выбрано локальное персистентное хранилище IndexedDB, что обеспечило накопление истории измерений и возможность повторного анализа без привлечения внешней инфраструктуры.

Ключевым практическим результатом является реализация конвейера автоматизированного сбора метрик, объединяющего несколько методик в рамках единой процедуры. В частности, реализованы аудит Lighthouse, сбор метрик через Web Performance API и проверка доступности, дополненные механизмами контроля прогресса и сохранения результатов. Полученные данные могут быть представлены как в виде детализированного отчета для отдельной платформы, так и в виде средств графического сравнения, что позволяет анализировать показатели сразу по нескольким направлениям и выявлять различия между альтернативными решениями.

Проведенная апробация прототипа на наборе тестируемых приложений подтвердила корректность основных сценариев управления платформами, воспроизводимость последовательности измерительных этапов и пригодность результатов для сравнительного анализа. Разработанная платформа может быть использована как основа для дальнейших исследований, включая расширение набора поддерживаемых метрик, автоматизацию серий экспериментов и углубленный анализ влияния архитектурных и конфигурационных факторов на наблюдаемые характеристики.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. *Черняевская А. С.* Платформы визуального программирования как будущее разработки веб-приложений // Студенческий вестник. — 2024. — 4—5(290). — С. 65—67. — EDN VAVPDC.
2. *Забержинский Б. Э.*, *Поляков Е. А.* Сравнительный анализ Low-code и No-code решений и стандартной разработки // Международный журнал гуманитарных и естественных наук. — 2024. — 10—2(97). — С. 93—96. — EDN HSNIGE.
3. *Максименкова Е. И.*, *Спрыгина Я. В.*, *Калязина С. Е.* Low-code платформы как инструмент создания корпоративных приложений // Молодежная Неделя Науки Института промышленного менеджмента, экономики и торговли: Сборник трудов всероссийской студенческой научно-учебной конференции. — Санкт-Петербург : Федеральное государственное автономное образовательное учреждение высшего образования ”Санкт-Петербургский политехнический университет Петра Великого”, 2022. — С. 39—42. — EDN XDUNSO.
4. *Chaudhary H. A. A.*, *Margaria T.* Integration of Micro-Services as Components in Modeling Environments for Low Code Development // Труды ИСП РАН. — 2021. — No. 4. — URL: `https://cyberleninka.ru/article/n/integration-of-micro-services-as-components-in-modeling-environments-for-low-code-development` (visited on 01/07/2025).
5. Репозиторий DIME. — 2025. — URL: `https://scce.gitlab.io/dime/content/developer-guide/extensions/generic-sibs.html` (дата обр. 15.05.2025) ; режим доступа свободный.
6. *Chaudhary H. A.*, *Margaria T.* Integrating External Services in DIME // Leveraging Applications of Formal Methods, Verification and Validation: 10th International Symposium on Leveraging Applications of Formal Methods, ISoLA 2021. — Rhodes, Greece : Springer International Publishing, 2021. — P. 41–54.

7. *Беляев А. С.*, *Десятириков Ф. А.* Open Source решения для выбора платформы разработки программного обеспечения // Международная конференция по мягким вычислениям и измерениям. Т. 1. — 2023. — С. 342—345. — EDN LKDPBL.

8. *Da Cruz M. A.*, *Paula H. T. de*, *Caputo B. P.*, [et al.]. OLP—A RESTful Open Low-Code Platform // Future Internet. — 2021. — Sept. — Vol. 13, no. 10. — P. 249.

9. Репозиторий OLP. — 2025. — URL: `https://github.com/heitor-lassarote/iolp` (дата обр. 25.05.2025) ; режим доступа свободный.

10. *Alfonso I.*, *Conrardy A.*, *Sulejmani A.*, [et al.]. Building BESSER: An Open-Source Low-Code Platform // International Conference on Business Process Modeling, Development and Support 2024. — Cham : Springer Nature Switzerland, 05/2024. — P. 203–212.

11. Репозиторий BESSER. — 2025. — URL: `https://github.com/BESSER-PEARL/BESSER` (дата обр. 15.05.2025) ; режим доступа свободный.

12. Репозиторий Jmix. — 2025. — URL: `https://github.com/jmix-framework/jmix` (дата обр. 10.05.2025) ; режим доступа свободный.

13. Репозиторий Pyro. — 2025. — URL: `https://github.com/pyro-ppl/pyro` (дата обр. 16.05.2025) ; режим доступа свободный.

14. *Mizambekov C.* Strategies for Reducing Costs in Custom Software Development: Leveraging Modern Technologies for Competitive Advantage // Journal of Current Science and Research Review. — 2024. — Dec. — Vol. 2, no. 03. — P. 99–112.